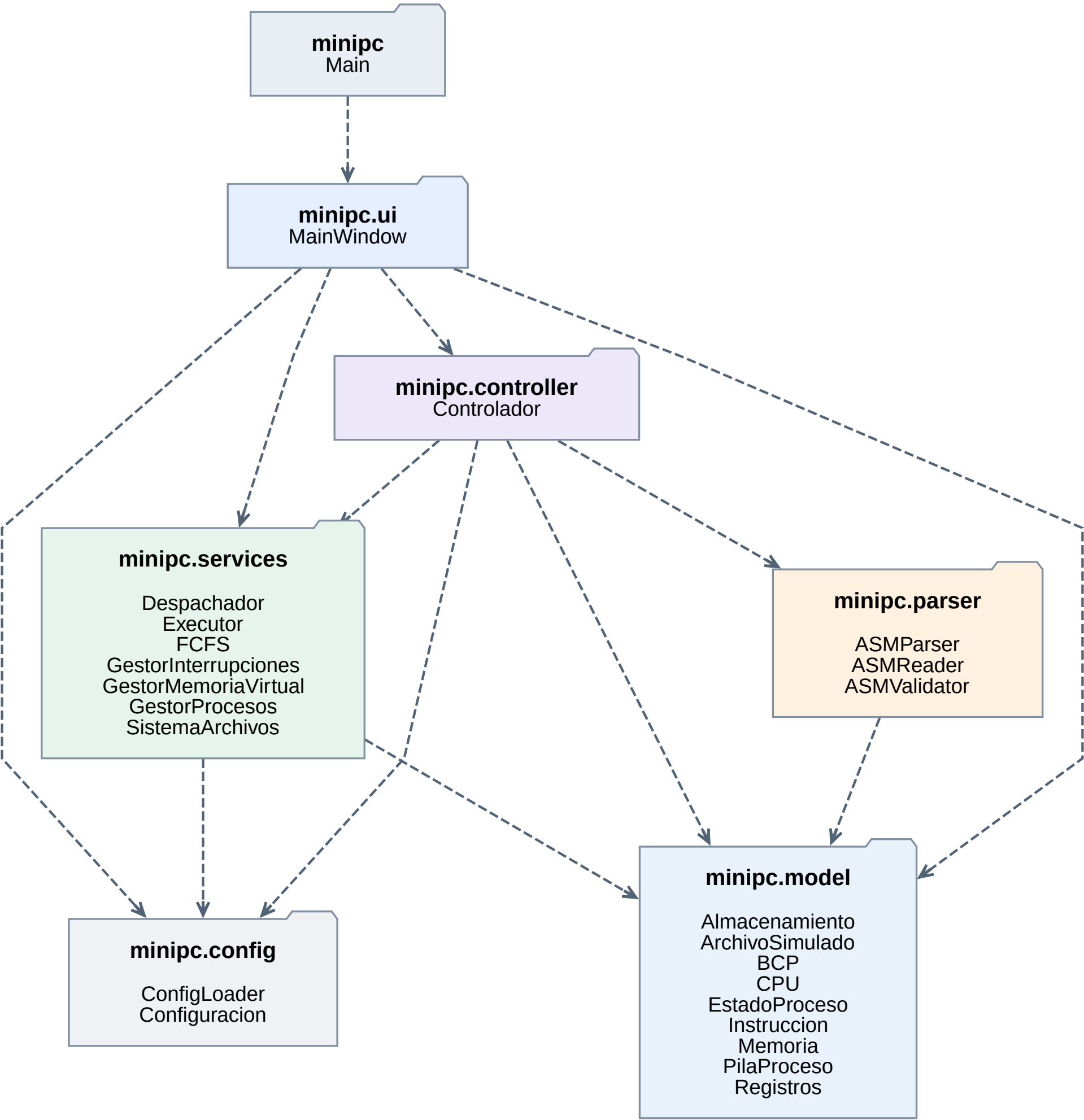


Mini PC — Diseño del sistema operativo

Diagrama de paquetes y explicación · Proyecto 1 · IC-6600
Owen Caleb Smith Cerdas · Carnet 2024083328



Dependencia UML: una flecha discontinua A -> B indica que A utiliza tipos o servicios de B.
Se muestran los 7 paquetes y sus 23 clases/enumeraciones principales; se omiten bibliotecas externas.

1. Arquitectura y responsabilidades

El sistema organiza la simulación en paquetes con responsabilidades separadas. El usuario carga programas ASM y observa su ejecución; Controlador coordina la admisión, la CPU, los procesos, la memoria y los servicios de entrada/salida. La interfaz presenta el estado y solicita operaciones al backend.

Organización de paquetes

- minipc: Main inicia la aplicación gráfica.
- minipc.ui: MainWindow muestra CPU, procesos, BCP, RAM, disco, archivos, monitor y estadísticas. Gestiona los botones y el temporizador automático.
- minipc.controller: Controlador conecta los componentes y dirige la carga, los ticks, la reactivación de suspendidos, el reinicio y la respuesta a errores.
- minipc.services: contiene FCFS, Despachador, Executor y los gestores de procesos, memoria virtual, interrupciones y archivos.
- minipc.model: representa CPU, registros, BCP, pila, instrucciones, estados, memoria y almacenamiento mediante atributos y estructuras tipadas.
- minipc.parser: ASMReader lee el archivo, ASMValidator comprueba su sintaxis y ASMParser construye las instrucciones y asigna sus pesos.
- minipc.config: ConfigLoader lee el TXT y Configuracion mantiene y valida los parámetros del sistema.

Decisiones de diseño

El BCP mantiene sus datos como atributos normales y el IR muestra la instrucción ASM original. La configuración selecciona algoritmo=FCFS, el algoritmo implementado en esta entrega. El diagrama muestra dependencias: una flecha de A hacia B significa que A utiliza tipos o servicios de B, no que B se ejecute después de A.

2. Procesos, planificación y ejecución

Cada ejecución admitida recibe un PID y un BCP propio. El BCP guarda estado, PC, IR, acumulador, registros, flag de comparación, pila, ubicación del programa, prioridad, enlaces, tiempos, archivos abiertos y motivo de error. Guardar y restaurar este contexto permite reanudar un proceso sin perder su avance.

Planificación FCFS

Los procesos PREPARADO se agregan al final de la cola Ready. Cuando la CPU queda libre, Despachador toma el primero y restaura su contexto. FCFS no utiliza quantum ni la prioridad almacenada para elegir al siguiente proceso. El proceso mantiene la CPU hasta bloquearse, finalizar o fallar.

Los estados representados son NUEVO, PREPARADO, EJECUCION, BLOQUEADO, FINALIZADO, PREPARADO_SUSPENDIDO y BLOQUEADO_SUSPENDIDO. Un proceso bloqueado en RAM vuelve al final de Ready cuando recibe la entrada de teclado. Si permanece en virtual, pasa a PREPARADO_SUSPENDIDO hasta poder regresar a RAM.

Ciclo de ejecución y pesos

Siguiente realiza una llamada de avance. En instrucciones de peso positivo, cada tick consumido equivale a un segundo simulado de CPU. Una instrucción de peso 3 conserva su PC hasta consumir tres ticks; después se aplica su efecto. Ejecutar usa un temporizador Swing de 1000 ms, permitiendo interactuar con la ventana mientras avanza. Pausar detiene el temporizador.

INT 09H tiene peso cero y bloquea por teclado sin consumir tiempo de CPU. Una interrupción puede guardar el contexto y entregar la CPU a otro proceso. El controlador evita incrementar el PC del nuevo proceso como si perteneciera a la instrucción anterior. El programa también finaliza al alcanzar naturalmente el final de sus instrucciones.

3. Memoria principal, virtual y almacenamiento

La RAM se divide al iniciar: las direcciones anteriores a inicioUsuario pertenecen al kernel y las restantes al área de usuario. El kernel no puede superar el 50% de RAM. La división permanece fija durante la simulación.

BCP y programas en RAM

Cada BCP reserva 24 posiciones consecutivas de kernel, una por atributo, incluido el motivo de error. La vista consulta sus valores actuales. Cada instrucción ocupa una posición de usuario. La asignación busca el primer bloque consecutivo libre que permita la carga.

Se admiten como máximo cinco procesos simultáneos en RAM, contando los bloqueados. El límite de procesos activos también depende del espacio físico para BCP: ram=256 e inicioUsuario=128 permiten cinco BCP de 24 posiciones. Para demostrar cinco procesos en RAM y otro suspendido se puede usar ram=512 e inicioUsuario=256, manteniendo maxProcesosEnRam=5.

Memoria virtual

Si no hay espacio o cupo en RAM, se admite el proceso en PREPARADO_SUSPENDIDO cuando hay capacidad en virtual y kernel. El BCP permanece en el kernel. GestorMemoriaVirtual registra pares [PID, dirección inicial virtual] y guarda las instrucciones. Cuando se libera RAM y un cupo, el controlador intenta reactivar suspendidos, restaura su ubicación y los incorpora a Ready.

El programa completo es la unidad de traslado; no se implementa paginación. Se rechaza un programa mayor que toda el área de usuario. No existe desalojo automático de bloqueados: el backend conserva una operación explícita de suspensión, que no se expone mediante un botón.

Disco simulado

El disco reserva primero el índice de nombres y direcciones, luego los archivos y al final la memoria virtual. Los programas importados se guardan también en disco; ocupan la suma de los pesos de sus instrucciones. Los archivos de texto ocupan espacio según la longitud de su contenido. Los bloques requieren posiciones consecutivas: la fragmentación puede impedir una asignación aunque existan posiciones libres dispersas.

4. Entrada/salida, protección y contabilidad

INT 09H guarda el contexto, bloquea al solicitante y permite ejecutar al siguiente. La entrada acepta enteros de 0 a 255 y se entrega al primero que espera teclado. INT 10H agrega el contenido de DX al monitor. INT 20H guarda el contexto final, termina el proceso y libera sus recursos.

Sistema de archivos

INT 21H utiliza AH para seleccionar crear (3CH), abrir (3DH), leer (4DH), escribir (40H) o eliminar (41H). DX contiene el nombre y AL se utiliza para el contenido. Cada BCP registra sus archivos abiertos; leer o escribir requiere apertura por ese proceso. Eliminar un archivo quita su referencia de todos los procesos. Al finalizar se limpian las referencias abiertas del proceso.

Protección y errores

ASMValidator concentra la validación sintáctica antes del parseo. Las condiciones dependientes del estado se comprueban al ejecutar: destino de salto dentro del programa, capacidad y disponibilidad de la pila, existencia de archivos y apertura por el proceso.

Ante un error de ejecución reconocido se guarda contexto y motivo, se marca FINALIZADO, se libera el programa y el BCP, se intenta reactivar suspendidos y se despacha el siguiente Ready. Monitor y estadísticas conservan el motivo. PARAM comprueba que todos sus valores caben antes de insertarlos; si desborda, la pila conserva el contenido anterior.

Estadísticas y reinicio

Tiempo CPU cuenta los segundos simulados consumidos por el proceso, incluidos los ticks anteriores a un fallo. Duración real mide desde la primera asignación de CPU hasta finalizar e incluye las esperas posteriores y pausas. Se muestran inicio y fin en HH:mm, contexto y resultado final.

El reinicio limpia CPU, RAM, BCP, procesos, colas, virtual, entradas pendientes, errores y control de ticks, y restablece los PID. Conservar archivos mantiene el disco simulado dentro de la misma aplicación; la memoria virtual siempre se limpia. Esta primera entrega utiliza FCFS y una única CPU.